

Aufgabe 1: Grundlagen: Fehlersuche (10 Punkte)

Finden Sie in jedem der folgenden „Python 3“-Programme den Fehler, der dazu führt, dass der Code nicht korrekt übersetzt bzw. ausgeführt werden kann, und erklären Sie, warum dies ein Fehler ist.

(a) (2 Punkte)

```
1|bestanden = true
2|if bestanden:
3|    print("Aufgabe bestanden")
4|else:
5|    print("Aufgabe nicht bestanden")
```

Zeile 1: NameError: name 'true' is not defined, Python ist Case Sensitive

(b) (2 Punkte)

```
1|stadt = ["Duisburg", "Essen", "Mülheim an der Ruhr", "Krefeld"]
2|einwohner = [495885, 582415, 170739]
3|for i in range(len(stadt)):
4|    print(stadt[i], "hat", einwohner[i], "Einwohner")
```

Zeile 2 bzw. Zeile 4: IndexError: list index out of range, die einwohner von Krefeld fehlen

(c) (2 Punkte)

```
1|def fakultaet(n):
2|    return n * fakultaet(n - 1)
3|n = 5
4|print("Fakultät =", fakultaet(n))
```

Zeile 2: RecursionError: maximum recursion depth exceeded; Abbruchbedingung fehlt

(d) (2 Punkte)

```
1|print("Countdown")
2|while i <= 10:
3|    print("{10 - i}")
4|    i += 1
```

Zeile 2: NameError: name 'i' is not defined; 'i' muss zunächst deklariert bzw. initialisiert werden)

(e) (2 Punkte)

```
1|a = 5
2|b = 0.6
3|print(a / int(b))
```

Zeile 3: ZeroDivisionError: division by zero; man darf nicht durch 0 teilen

Aufgabe 2: Grundlagen: Ausgabe.....(14 Punkte)

Geben Sie an, was die folgenden „Python 3“-Programme ausgeben.

(a) (2 Punkte)

```
1 vier = 2
2 zwei = 4
3 print(vier*vier == zwei)
```

True

(b) (2 Punkte)

```
1 a = 3
2 b = 4
3 for i in range(a,b):
4     print(i)
```

3

(c) (2 Punkte)

```
1 l = ["i", "j", "k"]
2 i = 0
3 while i < len(l):
4     print(l[i])
5     i += 1
```

**i
j
k**

(d) (4 Punkte)

```
1 a = "Rekursion macht allen Spaß"
2 print(a[16], a[17], a[18], a[19], a[5], a[9], a[2], a[18], a[16], a[4])
```

a l l e s k l a r

(e) (4 Punkte)

```
1 def k(i):
2     if i <= 5:
3         print(i)
4         k(i+1)
5 k(1)
```

**1
2
3
4
5**

Aufgabe 3: Run Length Encoding (32 Punkte)

Run Length Encoding (RLE) ist ein Algorithmus, mit dem Texte komprimiert werden können. Die Grundidee dabei ist, sich wiederholende Zeichen durch die Anzahl der Wiederholungen und das Zeichen zu ersetzen. Beispielsweise wird AAAAAB zu 5A1B komprimiert.

- (a) (18 Punkte) Schreiben Sie eine „Python 3“-Funktion `encode(text)`, welche den Eingabetext mit RLE komprimiert und anschließend zurückgibt. Schreiben Sie die Funktion so, dass dabei nicht mehr als 9 Zeichen auf einmal zusammengefasst werden; z.B. wird AAAAAAAAAAABBBBCDDDDDD zu 9A2A3B1C5D komprimiert.

```
1 def encode(text):
2     output=""
3     count=1
4     for i in range(len(text)):
5         if i < len(text)-1 and count < 9 and text[i] == text[i+1]:
6             count+=1
7         else:
8             output+=str(count)+text[i]
9             count=1
10    return output
```

- (b) (8 Punkte) Schreiben Sie eine „Python 3“-Funktion `decode(text)`, welche einen mit RLE komprimierten Text dekomprimiert und anschließend zurückgibt.

```
1 def decode(text):
2     output=""
3     for i in range(0, len(text), 2):
4         output+=text[i+1]*int(text[i])
5     return output
```

- (c) (4 Punkte) Während RLE den nötigen Speicher in vielen Fällen sehr effektiv reduziert, ist der Algorithmus nicht für alle Eingaben gut geeignet und kann die Länge im schlimmsten Fall sogar erhöhen. Beschreiben Sie, unter welchen Umständen dies der Fall ist.

Einzelne Zeichen werden in ihrer Länge verdoppelt (z.B. wird aus **A der Text **1A**). Falls im Eingabetext also verhältnismäßig viele einzelne Zeichen vorkommen, kann dies auch die Gesamtlänge vergrößern.**

- (d) (2 Punkte) Schreiben Sie eine „Python 3“-Funktion `checkRLE(text)`, welche für einen Eingabetext einen Wahrheitswert zurückgibt, der besagt, ob sich RLE lohnt oder nicht.

```
1 def checkRLE(text):  
2     return len(encode(text)) < len(text)
```

Aufgabe 4: Schimpfwortfilter.....(30 Punkte)

In dieser Aufgabe geht es darum, Chat-Nachrichten von Nutzer:innen auf unangebrachte Wörter (z.B. Beleidigungen, Schimpfwörter, Kraftausdrücke) mit „Python 3“ zu prüfen und diese durch neutrale Sternchen ("***") zu ersetzen. Wenn eine Nachricht zu viele oder zu schwere Verstöße enthält, soll die Nachricht stattdessen komplett gelöscht werden.

- (a) (12 Punkte) Zunächst müssen die Wörter eingelesen werden, welche zensiert werden sollen. Schreiben Sie dazu eine Funktion `readSwearwordData(filename)`. Die Funktion soll eine übergebene Textdatei einlesen, welche alle zu zensierenden Wörter enthält, daraus eine Python `list` erstellen und diese `list` zurückgeben.

Die Textdatei enthält in jeder Zeile zwei Informationen: ein zu zensierendes Wort und eine Ganzzahl, welche den Schweregrad der Unangemessenheit des Wortes (den Unangemessenheitswert) beschreibt (eine höhere Zahl steht dabei für ein schlimmeres Wort). Wort und Zahl sind jeweils durch ein Leerzeichen separiert. Hier ein Beispiel:

```
Fuck 1
Idiot 3
Vollpfosten 3
```

Ihre `list` soll pro Wort jeweils wiederum eine `list` mit zwei Einträgen enthalten: dem Wort selbst und dem Schweregrad. Für das obige Beispiel muss Ihr Ergebnis so aussehen: `[['Fuck', 1], ['Idiot', 3], ['Vollpfosten', 3]]`. Sollte die Datei nicht existieren oder nicht gelesen werden können, muss Ihre Funktion eine leere `list` zurückgeben.

Tipp: Um eine Zeile in einzelne Elemente aufzubrechen, können Sie auch die Funktion `split` verwenden. Beispiel:

```
line = "Hallo Welt 2"
line.split() # ergibt: ['Hallo', 'Welt', '2']
```

```
1 def readSwearwordData(filename):
2     try:
3         with open(filename, "r") as source:
4             result = []
5             lines = source.readlines()
6             for line in lines:
7                 tmp = line.split()
8                 word = tmp[0]
9                 severity = int(tmp[1])
10                result.append([word, severity])
11            return result
12     except:
13         return []
```

- (b) (16 Punkte) Schreiben Sie eine Funktion `checkText(message, swearwords)`, die einen String als ersten Übergabeparameter und eine list als zweiten Übergabeparameter bekommt. Die list ist so aufgebaut, wie es für das Ergebnis der ersten Teilaufgabe gefordert war, also beispielsweise `[['Fuck', 1], ['Idiot', 3], ['Vollpfosten', 3]]`.

Ihre Funktion soll nun für den übergebenen String Wort für Wort überprüfen, ob darin Wörter enthalten sind, die in der list mit unangemessenen Worten vermerkt sind. Dabei soll die Worterkennung unabhängig von der Groß- und Kleinschreibung funktionieren. Die Unangemessenheitswerte aller gefundenen Wörter sollen addiert werden. Ihre Funktion soll alle gefundenen unangebrachten Wörter im String durch die Zeichenkette "***" ersetzen und den zensierten Text zurückgeben, es sei denn, die Summe der Unangemessenheitswerte **ist größer als 7**. In dem Fall soll der String "Nachricht wurde gelöscht" zurückgegeben werden.

Hinweis: Sie dürfen davon ausgehen, dass nur Worte (mit einzelnen Leerzeichen dazwischen) in der Textnachricht enthalten sind. Es befinden sich also keine Satz- oder Sonderzeichen in der Nachricht.

```
1 def checkText(message, swearwords):
2     tmp = message.split()
3     impropriety = 0
4     editedMessage = ""
5     for word in tmp:
6         editedWord = word
7         for swearword in swearwords:
8             if (word.lower() == swearword[0].lower()):
9                 impropriety += swearword[1]
10                editedWord = "***"
11            editedMessage += editedWord + " "
12    if (impropriety > 7):
13        editedMessage = "Nachricht wurde gelöscht"
14    return editedMessage[:-1]
```

- (c) (2 Punkte) Schreiben Sie ein einzeliges Hauptprogramm, welches die Lösungen der Aufgabenteile a) und b) verwendet, um den Text "Du bist natürlich kein Idiot" mit der Schimpfwortliste in der Datei `swearwords.txt` zu überprüfen und den zensierten Text auf der Standardausgabe anzuzeigen.

```
1 print(checkText("Du bist natürlich kein Idiot", readSwearwordData("swearwords.txt")))
```

Aufgabe 5: Java (14 Punkte)

Für ein kleines norwegisches Möbelhaus *Aeki* sollen Sie eine Java-Klasse `Moebel` erstellen um Möbel zu repräsentieren. Die Möbel haben in diesem Fall nur zwei Merkmale: einen Namen und einen Preis. Versehen Sie die Klasse mit einem Konstruktor um Objekte der Klasse direkt beim Initialisieren mit Namen und Preis zu füllen. Erstellen Sie außerdem noch eine `toString`-Funktion, welche Ihr Möbelstück in verständlicher Textform zurückgibt. Ihr Code sollte in folgenden Rahmen passen:

```
1 public class Moebel{
2     public String name;
3     public float preis;
4
5     public Moebel(String n, float p){
6         name = n;
7         preis = p;
8     }
9
10    public String toString(){
11        return "Ich bin ein " + name + " und ich koste " + preis + " Euro.";
12    }
13
14    public static void main(String[] args) {
15        Moebel regal = new Moebel("Yllib", 40);
16        System.out.println(regal);
17    }
18 }
```